Hardware Shop – Online Shopping Application (Django + React)

Project documentation: plan, architecture, design, testing and delivery

Start date: Fri 23 Oct 2026 | Deadline: Mon 26 Oct 2026 | Stack: React, Django (DRF), PostgreSQL, Razorpay, Git

1. Project Overview

1.1 Scope (only what the task requires)

#	Requirement from the task	Covered in
1	Registration page	Auth module
2	Login with authentication	Auth module (JWT)
3	Forgot password	Auth module (email reset link)
4	Two user types: customer and admin	Role field on user
5	Display products, each with a list of spare parts	Catalog module
6	Customer: add to cart, checkout, pay with Razorpay	Cart, Orders, Payments
7	Invoice generated after checkout and sent to customer email	Invoices module
8	Admin: add products with multiple images (fields designed by us)	Catalog (admin)
9	Admin: view orders	Orders (admin)
10	Real-world scenario: many people buying the same product	Stock locking, Section 6.4
11	Professional UI, no errors, proper validations	Section 9
12	Planning, folder structure, naming, unit tests, manual testing, debugging, reusable code	Sections 3 to 12

1.2 Out of scope (deliberately not built)

Reviews and ratings, wishlist, coupons, product search filters beyond basic search, multiple addresses book, refunds UI, social login, real-time notifications, analytics dashboards, Docker/cloud deployment. These are not in the task, and leaving them out protects the deadline.

1.3 Assumptions

- Runs locally (the task says *locally*). Razorpay runs in **test mode** with test keys.
- Spare parts are sellable items with their own price and stock (the sample screenshots show price and add-to-cart on spare parts).
- Single currency (INR), GST shown as a flat configurable percentage on the invoice.
- Product data is entered by the admin; a small seed command provides demo data (Bosch GWS 600, INGCO RGH6528, and their spare parts).

2. Delivery Plan (3 working days + buffer)

Day	Date	Goal	Done when
1	Fri 23 Oct (today)	Planning, repo and environment setup, user model, auth APIs and pages	Register, login, forgot/reset password work end to end; auth unit tests pass
2	Sat 24 Oct	Catalog + spare parts, admin product form with multi-image upload, cart, checkout, Razorpay, invoice PDF and email	A customer can buy a product in Razorpay test mode and receives the invoice mail
3	Sun 25 Oct	Admin orders page, validations, UI polish, remaining unit tests, full manual test pass, bug fixing, README	All test cases in Section 11 pass; README tested from a clean clone
4	Mon 26 Oct	Buffer, final regression, push to Git, send reply email	Submitted before the deadline, ideally by midday

Priority order if time gets tight: auth, catalog with spare parts, cart, Razorpay payment, invoice email, admin add product, admin orders, UI polish. Never leave a half-working payment flow; cut polish first.

3. Architecture

3.1 High-level design

```
React SPA (Vite)  <--HTTPS/JSON + JWT-->  Django REST API  <--ORM-->
PostgreSQL
                                                |
                                                +--> Razorpay (create order,
verify signature)
                                                +--> SMTP / console email
(reset link, invoice)
                                                +--> Media storage (product
images, invoice PDFs)
```

- **Frontend:** React single-page app, React Router, Axios with an interceptor that attaches the JWT and refreshes it.
- **Backend:** Django with Django REST Framework, split into small apps by responsibility (accounts, catalog, cart, orders, payments, invoices).
- **Auth:** JWT (djangorestframework-simplejwt): short-lived access token plus refresh token.
- **Database:** PostgreSQL. All money values stored as `DecimalField`.
- **Payments:** Razorpay Orders API plus server-side signature verification.
- **Email:** Django email backend. Console backend in development; SMTP (e.g. Gmail app password) for the real invoice test.
- **Invoice PDF:** generated server-side (ReportLab) and attached to the email.

3.2 Layered backend design (keeps code reusable)

Each app follows the same layers so logic is never buried in views:

- `models.py` – data and simple data rules
- `serializers.py` – validation and input/output shape
- `services.py` – business logic (e.g. `create_order_from_cart`, `verify_payment`, `generate_invoice`)
- `views.py` – thin request/response handling that calls services
- `permissions.py`, `urls.py`, `tests/`

Shared code (pagination, permissions such as `IsAdminRole` and `IsCustomerRole`, common exceptions, timestamp base model) lives in a `common` app.

3.3 Core flows

Registration and login: the user submits the form, the API validates and creates the user (role = customer), then the user logs in and receives access and refresh tokens. The React `AuthContext` stores them and exposes the user and role. Admin accounts are created with `createsuperuser` or the seed command with `role=admin`.

Forgot password: user enters their email, the API always responds with a generic success message (no account enumeration) and emails a link `FRONTEND_URL/reset-password/<uid>/<token>/` built with Django's `PasswordResetTokenGenerator`. The reset page posts the uid, token and new password; the token is single-use and expires (default 1 hour).

Checkout and payment (the critical flow):

1. Customer clicks Checkout and enters shipping details.
2. `POST /orders/checkout/` runs in one database transaction: it locks the stock rows, validates quantities, creates the `Order` (status `PENDING_PAYMENT`) and `OrderItem` snapshots, reserves stock, creates a Razorpay order and stores `razorpay_order_id` in a `Payment` row.
3. The frontend opens the Razorpay Checkout popup with the returned `razorpay_order_id` and public key.
4. On success, the frontend sends `razorpay_payment_id`, `razorpay_order_id` and `razorpay_signature` to `POST /payments/verify/`.
5. The server verifies the HMAC signature with the Razorpay secret. If valid: mark `Payment` captured and `Order` as `PAID`, clear the cart, generate the invoice PDF, and email it. If invalid or the user closes the popup: mark `FAILED` and release the reserved stock.
6. Verification is **idempotent**: calling it twice never creates two invoices or deducts stock twice.

4. Folder Structure

```
hardware-shop/
├── README.md
├── .gitignore
├── backend/
│   ├── manage.py
│   ├── requirements.txt
│   ├── .env.example
│   ├── config/                    # project package
│   │   ├── settings/
```

```
|   |   |   ├── base.py
|   |   |   ├── dev.py
|   |   |   └── __init__.py
|   |   ├── urls.py
|   |   └── wsgi.py
|   ├── common/                    # shared helpers
|   |   ├── models.py              # TimeStampedModel
|   |   ├── permissions.py         # IsAdminRole, IsCustomerRole
|   |   ├── pagination.py
|   |   └── exceptions.py
|   ├── accounts/
|   |   ├── models.py  serializers.py  services.py  views.py  urls.py
|   |   └── tests/ (test_models.py, test_auth_api.py, test_password_reset.py)
|   ├── catalog/
|   |   ├── models.py              # Product, ProductImage, SparePart
|   |   ├── serializers.py  views.py  urls.py
|   |   ├── management/commands/seed_catalog.py
|   |   └── tests/
|   ├── cart/
|   |   ├── models.py  serializers.py  services.py  views.py  urls.py
|   |   └── tests/
|   ├── orders/
|   |   ├── models.py              # Order, OrderItem
|   |   ├── services.py            # checkout, stock reservation
|   |   └── tests/
|   ├── payments/
|   |   ├── models.py              # Payment
|   |   ├── razorpay_client.py     # thin wrapper around the SDK
|   |   ├── services.py            # create_razorpay_order, verify_payment
|   |   └── tests/                 # Razorpay mocked
|   ├── invoices/
|   |   ├── models.py              # Invoice
|   |   ├── services.py            # generate_pdf, send_invoice_email
|   |   ├── templates/invoices/invoice_email.txt
|   |   └── tests/
|   └── media/                     # uploads (git-ignored)
└── frontend/
    ├── package.json  vite.config.js  .env.example
    └── src/
        ├── api/                   # axiosClient.js, authApi.js,
productApi.js, cartApi.js, orderApi.js
        ├── context/               # AuthContext.jsx, CartContext.jsx
        ├── hooks/                 # useAuth.js, useCart.js
        ├── components/            # Navbar, ProductCard, SparePartCard,
QuantityInput, ProtectedRoute, FormField, Loader
        ├── pages/
```

```
        |   ├── auth/              # RegisterPage, LoginPage,
ForgotPasswordPage, ResetPasswordPage
        |   ├── shop/              # ProductListPage, ProductDetailPage,
CartPage, CheckoutPage, OrderSuccessPage, MyOrdersPage
        |   └── admin/             # AdminProductsPage, AdminProductFormPage,
AdminOrdersPage
        ├── routes/                # AppRoutes.jsx
        ├── utils/                 # validators.js, formatCurrency.js
        └── styles/
```

5. Naming Conventions

Area	Convention	Example
Python modules, functions, variables	`snake_case`	`create_order_from_cart()`
Python classes	`PascalCase`	`OrderItem`, `ProductSerializer`
Constants and settings	`UPPER_SNAKE_CASE`	`GST_PERCENT`, `RAZORPAY_KEY_ID`
Django apps	lowercase plural noun	`orders`, `payments`
DB tables	Django default (`app_model`)	`orders_orderitem`
Model fields	`snake_case`; booleans start with `is_` / `has_`	`is_active`
Serializers	`<Model><Purpose>Serializer`	`ProductDetailSerializer`
Views	`<Thing><Action>View`	`CheckoutView`
REST URLs	lowercase, plural nouns, hyphen-free where possible	`/api/v1/spare-parts/`
Tests	`test_<behaviour>` in `test_<module>.py`	`test_checkout_fails_when_out_of_stock`
React components and pages	`PascalCase.jsx`	`ProductCard.jsx`

Area	Convention	Example
Hooks	`useCamelCase`	`useCart`
JS variables and functions	`camelCase`	`handlePayment`
API modules	`<resource>Api.js`	`orderApi.js`
Git branches	`feature/<name>`, `fix/<name>`	`feature/razorpay-checkout`
Commit messages	imperative, prefixed	`feat: add cart API`, `fix: stock check on checkout`, `test: add auth tests`

6. Data Model Design

All models inherit `TimeStampedModel` (`created_at`, `updated_at`) from `common`.

6.1 Entities

accounts.User (custom user, set `AUTH_USER_MODEL` before the first migration)

Field	Type	Rules
email	EmailField	unique, used as the login identifier
first_name, last_name	CharField	required
phone	CharField(15)	required, validated (10-digit Indian mobile, optional +91)
role	CharField	`customer` (default) or `admin`
is_active, is_staff	Boolean	Django defaults
password	hashed	Django validators (min length 8, not common, not all numeric)

catalog.Product

Field	Type	Notes
name	CharField(200)	e.g. Bosch Professional Angle Grinders - GWS 600
slug	SlugField	unique, auto-generated

Field	Type	Notes
model_code	CharField(50)	e.g. 060137505F, RGH6528
brand	CharField(100)	Bosch, INGCO
description	TextField	
mrp	Decimal(10,2)	original price (shown struck through)
price	Decimal(10,2)	selling price, must be ⩽ mrp
stock	PositiveIntegerField	
is_active	Boolean	hides product without deleting it

catalog.ProductImage: `product` (FK, related_name `images`), `image` (ImageField), `is_primary` (Boolean), `sort_order` (int). Allowed formats JPG/PNG/WebP, max 2 MB each.

catalog.SparePart: `product` (FK, related_name `spare_parts`), `name` (e.g. Armature (220-240V)), `part_code`, `description`, `price`, `stock`, `image` (optional), `is_active`.

cart.Cart: `user` (OneToOne). **cart.CartItem:** `cart` (FK), `product` (FK, nullable), `spare_part` (FK, nullable), `quantity` (⩾ 1). A database `CheckConstraint` guarantees **exactly one** of `product` / `spare_part` is set, and unique constraints stop duplicate lines (the same item increases quantity instead).

orders.Order

Field	Notes
order_number	unique, human-readable, e.g. `ORD-20261024-0001`
user	FK to User (PROTECT)
status	`PENDING_PAYMENT`, `PAID`, `FAILED`, `CANCELLED`
ship_name, ship_phone, ship_address_line, ship_city, ship_state, ship_pincode	shipping snapshot; pincode validated as 6 digits
subtotal, tax_amount, total_amount	Decimal(12,2)

orders.OrderItem: `order` (FK), `product` / `spare_part` (nullable FKs, SET_NULL), plus **snapshot fields** `item_name`, `item_code`, `unit_price`, `quantity`, `line_total`. Snapshots mean a later price change or product deletion never alters past orders or

invoices.

payments.Payment: `order` (FK), `razorpay_order_id` (unique), `razorpay_payment_id`, `razorpay_signature`, `amount`, `currency` (INR), `status` (`CREATED`, `CAPTURED`, `FAILED`).

invoices.Invoice: `order` (OneToOne), `invoice_number` (unique, e.g. `INV-2026-00001`), `pdf_file` (FileField), `issued_at`, `emailed_at` (null until sent).

Password reset needs no table: Django's token generator signs the token with the user's password hash, so it becomes invalid as soon as the password changes.

6.2 Relationships

```
User 1──1 Cart 1──* CartItem *──1 Product / SparePart
User 1──* Order 1──* OrderItem
Order 1──* Payment
Order 1──1 Invoice
Product 1──* ProductImage
Product 1──* SparePart
```

6.3 Indexes and constraints

Index `Product.slug`, `Product.is_active`, `Order.user`, `Order.status`, `Order.created_at`, `Payment.razorpay_order_id`. Check constraints: `price <= mrp`, `quantity >= 1`, one-of product/spare_part on `CartItem`.

6.4 Many customers buying the same product (concurrency)

The task explicitly asks to keep this real-world scenario in mind. The design:

- Checkout runs inside `transaction.atomic()`.
- Stock rows are read with `select_for_update()` (row-level lock, rows locked in a consistent order by id to avoid deadlocks), so two simultaneous buyers cannot both take the last unit.
- Stock is **reserved** when the order is created and decremented with an `F()` expression, never read-modify-write in Python.
- If payment fails, is cancelled, or never completes (pending orders older than 30 minutes are released by a small management command), the reserved stock is returned.
- Payment verification is idempotent (status checked under lock), so retries and double clicks are harmless.
- A unit test spawns concurrent checkouts for stock = 1 and asserts only one succeeds.

7. REST API Design

Base path `/api/v1/`. JSON everywhere. Errors use a consistent shape: `{"detail": "...", "errors": {"field": ["message"]}}`.

Module	Method and path	Access	Purpose
Auth	`POST /auth/register/`	public	Create customer account
	`POST /auth/login/`	public	Returns access and refresh tokens and user info
	`POST /auth/token/refresh/`	public	New access token
	`POST /auth/forgot-password/`	public	Email reset link
	`POST /auth/reset-password/`	public	Set new password using uid and token
	`GET /auth/me/`	logged in	Current user and role
Catalog	`GET /products/`	public	List active products (paginated, `?search=`)
	`GET /products/<slug>/`	public	Product detail with images and spare parts
	`POST /admin/products/`	admin	Create product (multipart, multiple images)
	`PUT/PATCH/DELETE /admin/products/<id>/`	admin	Edit, or deactivate
	`POST /admin/products/<id>/spare-parts/`	admin	Add spare part
	`PUT/DELETE /admin/spare-parts/<id>/`	admin	Edit or remove spare part
Cart	`GET /cart/`	customer	View cart with totals
	`POST /cart/items/`	customer	Add `{product_id` or `spare_part_id, quantity}`
	`PATCH /cart/items/<id>/`	customer	Change quantity

Module	Method and path	Access	Purpose
	`DELETE /cart/items/<id>/`	customer	Remove line
Orders	`POST /orders/checkout/`	customer	Create order and Razorpay order
	`GET /orders/`	customer	My orders
	`GET /orders/<order_number>/`	customer	Order detail
	`GET /admin/orders/`	admin	All orders (filter by status, search by order number or email)
	`GET /admin/orders/<order_number>/`	admin	Order detail
Payments	`POST /payments/verify/`	customer	Verify signature, finalize order
	`POST /payments/failed/`	customer	Mark attempt failed, release stock
Invoices	`GET /orders/<order_number>/invoice/`	owner or admin	Download invoice PDF

Customers can only ever read their own orders and invoices (queryset filtered by `request.user`); admin endpoints require `IsAdminRole`.

8. Frontend Design

8.1 Pages and routes

Route	Page	Access
`/register`, `/login`, `/forgot-password`, `/reset-password/:uid/:token`	Auth pages	public
`/`	Product list (cards with image, name, model code, price, struck-through MRP, "Save ₹X" badge)	public

Route	Page	Access
`/products/:slug`	Product detail: image gallery, description, **spare parts list** with price and Add to cart	public (add to cart needs login)
`/cart`	Cart with quantity update, remove, totals	customer
`/checkout`	Shipping form and Pay with Razorpay	customer
`/order-success/:orderNumber`	Confirmation and invoice download	customer
`/my-orders`	Order history with invoice links	customer
`/admin/products`, `/admin/products/new`, `/admin/products/:id/edit`	Product management with multi-image upload and spare parts editor	admin
`/admin/orders`	All orders table	admin

`ProtectedRoute` checks login and role; a customer opening an admin URL is redirected.

8.2 UI guidelines (professional look without extra libraries to learn)

- One consistent palette and typography, responsive grid for product cards, a sticky navbar with cart badge.
- Use a single component library or plain CSS modules, not both. Suggested: plain CSS with CSS variables, or Bootstrap for speed.
- Every form shows inline field errors, disabled submit button while loading, toast messages for success and failure, loading and empty states on lists.
- Currency formatted with one helper (`formatCurrency`) in the `₹1,23,456.00` style.

8.3 Reusable pieces

`FormField` (label, input, error), `ProductCard`, `SparePartCard`, `QuantityInput`, `ProtectedRoute`, `Loader`, a shared `axiosClient`, and `validators.js` used by every form.

9. Validation Rules

Validate on **both** sides: React for instant feedback, Django as the source of truth.

Form	Rules
Register	First name required; valid email, unique (case-insensitive); phone 10 digits starting 6-9; password min 8 chars, not entirely numeric, not common; confirm password matches
Login	Email and password required; wrong credentials give one generic message
Forgot / reset password	Valid email format; new password follows the same rules; expired or used token shows a clear message
Add to cart	Quantity integer ⩾ 1 and ⩽ available stock
Checkout	Name, phone, address line, city, state required; pincode exactly 6 digits
Admin product	Name, brand, model code, price, MRP, stock required; price ⩽ MRP; price and stock not negative; images JPG/PNG/WebP, max 2 MB each, at least 1 image, exactly one primary
Spare part	Name, price, stock required; price not negative

10. Security Checklist

- Passwords hashed (Django default); no plaintext anywhere.
- Secrets (Django secret key, DB password, Razorpay key and secret, email credentials) only in `.env`, never committed. Commit `.env.example`.
- Razorpay **secret never reaches the frontend**; only the public key id does.
- Payment trusted only after server-side signature verification. Amount is always computed on the server from the database, never taken from the client.
- CORS limited to the frontend origin; CSRF not needed for token auth but keep Django defaults for the admin site.
- Role checks on every admin endpoint; ownership checks on orders and invoices.
- Rate limit login and forgot-password using DRF throttling.
- Generic response on forgot-password so emails cannot be enumerated.

11. Testing Plan

11.1 Unit and API tests (Django `TestCase` / DRF `APITestCase`, run with `python manage.py test`)

App	Test cases
accounts	Register succeeds; duplicate email rejected; weak password rejected; invalid phone rejected; login succeeds and returns tokens; wrong password rejected; `/auth/me/` requires token; forgot-password returns the same response for unknown email; reset works with a valid token; expired or reused token rejected
catalog	Product list only shows active items; detail includes images and spare parts; non-admin cannot create product; admin creates product with multiple images; price greater than MRP rejected; invalid image type rejected; spare part attaches to the right product
cart	Add product; add spare part; adding same item twice increases quantity; quantity over stock rejected; update and delete line; one customer cannot see another's cart; exactly-one-of constraint enforced
orders	Checkout creates order and item snapshots with correct totals; stock reserved; empty cart rejected; out-of-stock rejected; **concurrent checkout of last unit yields one success**; customer cannot read another customer's order; admin can list all orders
payments	Razorpay client mocked: valid signature marks order `PAID`; invalid signature rejected; duplicate verify is idempotent; failed payment releases stock
invoices	Invoice number unique and sequential; PDF file is created; email is sent with an attachment (`mail.outbox`); only owner or admin can download

Target: every service function has at least one success test and one failure test. Razorpay and email are always mocked in automated tests.

11.2 Manual test checklist (run on Sun 25 Oct, repeat quickly on Mon 26 Oct)

1. Register a new customer, confirm validation messages for each bad input, log in, log out, confirm login with a wrong password fails.
2. Forgot password: request link, open it, set a new password, confirm the old password fails and the link cannot be reused.
3. Log in as admin, add a product with 3 images and 2 spare parts; confirm everything appears on the storefront.
4. As customer, open the product, see spare parts, add the product and a spare part to the cart, change quantities, remove one line.
5. Checkout with invalid shipping data (errors shown), then valid data, then pay with a Razorpay **test card/UPI**; confirm the success page, order status `PAID`, stock reduced, cart emptied.
6. Confirm the invoice PDF downloads and the invoice **email arrives** with the attachment and correct totals.
7. Close the Razorpay popup without paying: order becomes failed and stock is released.
8. Open two browsers and buy the last unit at the same time; only one order succeeds.
9. Customer opens an admin URL (blocked); customer tries another customer's order URL (blocked).
10. Admin sees all orders with customer, amount and status.
11. Refresh on every page while logged in (session persists); expired access token refreshes silently.
12. Check layout at desktop and mobile width.

Keep a short log of each bug found and the fix; it helps in the interview discussion.

12. Setup and Run Instructions (to be included in README)

Prerequisites: Python 3.11+, Node 18+, PostgreSQL 14+, Git.

```
# Backend
cd backend
python -m venv venv && source venv/bin/activate   # Windows:
venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env        # fill values
# create database first: CREATE DATABASE hardware_shop;
python manage.py migrate
python manage.py seed_catalog
python manage.py createsuperuser   # role=admin
python manage.py runserver

# Frontend
cd frontend
npm install
cp .env.example .env
npm run dev
```

Backend `.env` keys: `DJANGO_SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT`, `FRONTEND_URL`, `CORS_ALLOWED_ORIGINS`, `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `EMAIL_BACKEND`, `EMAIL_HOST`, `EMAIL_PORT`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`, `DEFAULT_FROM_EMAIL`, `GST_PERCENT`.

Frontend `.env` keys: `VITE_API_BASE_URL`, `VITE_RAZORPAY_KEY_ID`.

Python packages: django, djangorestframework, djangorestframework-simplejwt, django-cors-headers, psycopg2-binary, python-decouple (or django-environ), Pillow, razorpay, reportlab.

npm packages: react, react-router-dom, axios, plus one toast library (e.g. react-hot-toast).

13. Git Workflow

- `main` always runnable; work on short feature branches and merge when tests pass.
- Suggested commit milestones: project skeleton, user model and auth, catalog and spare parts, admin product form, cart, checkout and Razorpay, invoice and email, admin orders, tests, UI polish, README.
- Small, frequent commits with the message style from Section 5. Never commit `.env`, `venv`, `node_modules` or `media/`.

14. Risks and Mitigations

Risk	Mitigation
Razorpay setup delays	Create a test-mode account and keys on Day 1; build payments against a mocked client first
Email not arriving (SMTP setup)	Use console backend during development; set up Gmail app password early on Day 2
Running out of time	Follow the priority order in Section 2; skip polish, not correctness
Concurrency bugs	Keep stock logic in one service function with its own tests
Not able to explain the code	Keep the layered structure, comment the checkout and payment services, and re-read the flow in Section 3.3 before the interview

15. Final Submission Checklist

- [ ] All features in Section 1.1 work end to end
- [ ] `python manage.py test` passes with no failures
- [ ] Manual checklist (Section 11.2) completed and bugs fixed
- [ ] README tested from a clean clone
- [ ] `.env.example` present, no secrets committed
- [ ] Seed data loads and admin credentials are written in the README
- [ ] Code pushed to Git, repository link ready
- [ ] Reply email sent before the deadline (Mon 26 Oct)

16. Reply Email Template

Subject: Submission – Online Shopping Application (Django-React) – [Your Name]

Hello [Recruiter/Team name],

I have completed the Online Shopping Application task. Please find the details below.

- Repository: [Git link]
- Stack: React, Django REST Framework, PostgreSQL, Razorpay (test mode), Git

- Implemented: registration, login with JWT authentication, forgot/reset password, customer and admin roles, product catalog with spare parts, cart, checkout and Razorpay payment, invoice generation with email to the customer, admin product creation with multiple images, and admin order view.
- Real-world handling: stock locking so multiple customers can buy the same product safely, server-side payment verification, and validations on both frontend and backend.
- Testing: unit and API tests included (`python manage.py test`) and a full manual test pass.
- Setup steps, environment variables and demo credentials are in the README.

Please let me know if you need anything else. Thank you for the opportunity.

Regards, [Your Name] [Phone]